

JJBike

1. Objetivo.

Construir um armazém de dados para que o administrador da empresa fictícia JJBike possa compreender seu negócio.

2. Modelagem.

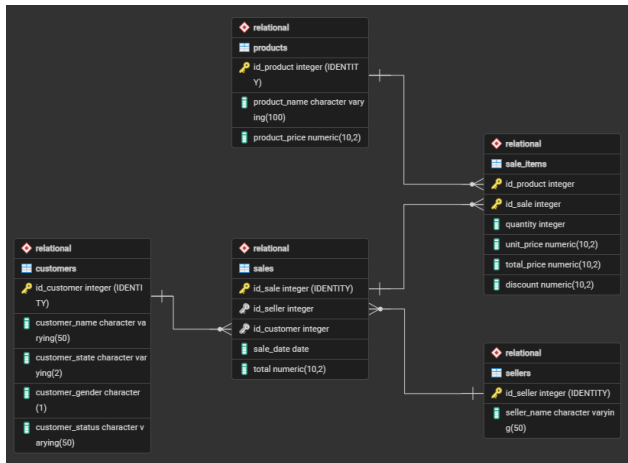
Para criar o modelo da estrutura do banco de dados analítico (OLAP) com base no banco de dados transacional (OLTP), utilizando o modelo Star Schema (modelo mais comum para Business Intelligence), foram feitas as seguintes transformações:

1. Para criar o modelo da estrutura do banco de dados analítico (OLAP) considerando o banco de dados transacional (OLTP), o assunto que se quer analisar (o fato) e as dimensões são os diversos pontos de vista sobre o qual se quer analisar o fato (as dimensões), utilizando o modelo Star Schema (modelo mais comum para Business Intelligence), foram feitas as seguintes transformações:

Dimensão	Atributos	Tabela de origem
dim_product (o quê?)	product_key (SK)	-
	id_product	product
	product_name	product
	validity_start_date (versionamento)	-
	validity_end_date (versionamento)	-
dim_seller (quem?)	seller_key (SK)	-
	id_seller	seller
	seller_name	seller
	validity_start_date (versionamento)	-
	validity_end_date (versionamento)	-
dim_customer (quem?)	customer_key (SK)	-
	id_customer	customer
	customer_name	customer
	customer_state	customer

	customer_gender	customer
	customer_status	customer
	validity_start_date (versionamento)	-
	validity_end_date (versionamento)	-
dim_time (quando?)	time_key (SK)	-
	date	-
	time_day	-
	time_month	-
	time_year	-
	time_weekday	-
	time_quarter	-

Fato	Tabela consolidada	Granularidade	Atributos	Tabela de origem
fact_sales	sales	Item de produto por venda.	sale_key (SK)	-
			seller_key (FK)	dim_seller
			customer_key (FK)	dim_customer
			product_key	dim_product
			time_key (FK)	dim_time
			quantity	sale_items
			unit_price	sale_items
			total_price	sale_items
			discount	sale_items



3. pgAdmin 4.

No pgAdmin 4, foi criado o banco de dados 'JJBike'.

Obs.: No desenvolvimento do código SQL, será utilizado o padrão de nomenclatura Snake Case.

4. Ambiente Transacional (OLTP) - pgAdmin 4.

4.1. Criação do Schema e das tabelas transacionais.

Foi criado o schema Relational, que serve como origem dos dados transacionais da empresa JJBike. Dentro dele, foram criadas cinco tabelas operacionais por meio do arquivo 1. Scripts_Create_Table_Relational.sql:

- **sellers:** Armazena os dados dos vendedores. Sua chave primária, `id_seller`, é gerada automaticamente pela cláusula `GENERATED ALWAYS AS IDENTITY`. O atributo `seller_name` recebe a restrição `NOT NULL`, garantindo que nenhum vendedor seja cadastrado sem nome;
- **products:** Armazena o catálogo de produtos. A chave primária `id_product` é gerada automaticamente. Os atributos `product_name` e `product_price` são definidos como `NOT NULL`, garantindo que todo produto tenha nome e preço registrados;
- **customers:** Armazena o cadastro de clientes. A chave primária `id_customer` é gerada automaticamente. Os atributos `customer_name`, `customer_state`, `customer_gender` e `customer_status` são todos `NOT NULL`, assegurando que o perfil de cada cliente esteja sempre completo;
- **sales:** Representa o cabeçalho de cada venda. A chave primária `id_sale` é gerada automaticamente. As colunas `id_seller` e `id_customer` são Foreign Keys `NOT NULL` que referenciam, respectivamente, as tabelas `sellers` e `customers`, impedindo que uma venda seja registrada sem vendedor ou cliente associado. O atributo `sale_date` registra a data em que a venda ocorreu, e `total` armazena o valor total da transação, ambos obrigatórios;
- **sale_items:** Tabela de ligação N:M entre `sales` e `products`. Sua chave primária é composta por `id_product` e `id_sale`. Por ser uma tabela de ligação, foram aplicadas duas restrições de integridade referencial distintas:
 - `ON DELETE RESTRICT` em `id_product`: impede a exclusão de um produto em `Relational.products` enquanto ele estiver referenciado em algum item de venda, evitando que relatórios históricos percam a descrição do produto vendido;

- ON DELETE CASCADE em id_sale: garante que, ao se excluir um registro em Relational.sales, todos os seus itens correspondentes em sale_items sejam automaticamente removidos, eliminando registros órfãos de vendas inexistentes;
- Os atributos quantity (quantidade), unit_price (preço unitário), total_price (valor total do item) e discount (desconto aplicado) são todos NOT NULL, assegurando a integridade das métricas que posteriormente alimentarão a tabela fato.

A cláusula GENERATED ALWAYS AS IDENTITY foi utilizada em todas as tabelas para geração automática das Primary Keys. Todas as colunas, exceto as próprias Primary Keys e validity_end_date nas dimensões, foram definidas com a restrição NOT NULL, garantindo a integridade dos dados no nível do banco de dados. A restrição padrão ao definir uma FK sem especificar uma ação de exclusão ou atualização é o NO ACTION, que bloqueia a operação e gera um erro caso se tente deletar um registro pai com dependentes na tabela filha, impedindo a criação de dados órfãos.

4.2. Inserção de dados nas tabelas transacionais.

Após a criação das tabelas, foram realizadas as inserções de dados por meio dos arquivos 2. Insert_Into_Customers.sql, 3. Insert_Into_Products.sql, 4. Insert_Into_Sellers.sql, 5. Insert_Into_Sales.sql e 6. Insert_Into_SaleItems.sql. Cada arquivo executa comandos INSERT INTO na respectiva tabela do schema Relational. Como as Primary Keys são gerenciadas por GENERATED ALWAYS AS IDENTITY, os scripts não informam esses valores explicitamente, o banco os atribui automaticamente e de forma sequencial a cada registro inserido.

5. Ambiente Analítico (OLAP) – pgAdmin 4.

5.1. Criação do Schema e das Tabelas Dimensionais.

Foi criado o schema Dimensional, que representa o ambiente analítico (Data Warehouse) estruturado em modelo Star Schema. Dentro dele, foram criadas as dimensões dim_seller, dim_customer, dim_product, dim_time e a tabela fato fact_sales por meio do arquivo 1. Scripts_Create_Table_Dimensional.sql.

A cláusula GENERATED ALWAYS AS IDENTITY foi utilizada para gerar automaticamente as Surrogate Keys (SKs) de todas as dimensões e da tabela fato, servindo como chave primária interna do Data Warehouse. Todas as colunas foram definidas com NOT NULL, com exceção das próprias Primary Keys e do campo validity_end_date nas dimensões com SCD Tipo 2, esse campo permanece NULL enquanto o registro estiver ativo, sendo preenchido apenas quando uma nova versão do registro é criada.

As tabelas dimensionais possuem a seguinte estrutura de atributos:

- **dim_seller:** seller_key (SK gerada automaticamente, PK interna do DW), id_seller (ID de origem da tabela Relational.sellers, preservado para rastreabilidade), seller_name (nome do vendedor), validity_start_date (data em que este registro passou a ser válido) e validity_end_date (data em que este registro foi encerrado; NULL indica registro ativo);
- **dim_customer:** customer_key (SK gerada automaticamente, PK interna do DW), id_customer (ID de origem), customer_name, customer_state, customer_gender, customer_status, validity_start_date e validity_end_date. Por possuir múltiplos atributos rastreáveis, é a dimensão com maior potencial de versionamento SCD Tipo 2;

- **dim_product:** product_key (SK), id_product (ID de origem), product_name, validity_start_date e validity_end_date;
- **dim_time:** time_key (SK), time_date (data completa), time_day (dia do mês), time_month (mês), time_year (ano), time_weekday (dia da semana, como número inteiro) e time_quarter (trimestre). Esta dimensão não possui SCD pois o tempo é imutável;
- **fact_sales:** sale_key (SK, PK da fato), seller_key, customer_key, product_key e time_key (FKs que referenciam as respectivas dimensões, todas NOT NULL), além das métricas quantity, unit_price, total_price e discount, também NOT NULL.

5.2. População da dimensão tempo.

A dimensão dim_time foi populada por meio do arquivo 2. Insert_Into_dim_time.sql. A estratégia utilizada foi a geração programática de um intervalo contínuo de datas de 1º de janeiro de 1970 a 31 de dezembro de 2049, cobrindo 29.220 dias (índices 0 a 29.219). O script é composto por três estruturas encadeadas:

- O bloco FROM utiliza a função GENERATE_SERIES(0, 29219) para produzir uma sequência de inteiros que, somados à data base '1970-01-01'::DATE, geram a coluna temporária datum com todas as datas do intervalo. Esse conjunto é agrupado por SEQUENCE.DAY para garantir que cada data apareça exatamente uma vez;
- O bloco SELECT fatia cada valor de datum com a função EXTRACT, derivando os atributos analíticos: time_day (dia do mês), time_month (mês), time_year (ano), time_weekday (dia da semana numérico, retornado pelo parâmetro dow, 0 para domingo, 6 para sábado) e time_quarter (trimestre). O próprio datum é atribuído ao campo time_date;
- O bloco ORDER BY 1 garante que os registros sejam inseridos em ordem cronológica crescente, do dia mais antigo para o mais recente, o que mantém a integridade sequencial da dimensão tempo.

5.3. Carga de dados do OLTP para o OLAP (SCD Tipo 2).

As cargas de dados do ambiente transacional para o analítico foram realizadas por meio do arquivo 3. Script.sql. O roteiro foi estruturado em fases sequenciais para demonstrar o funcionamento completo do mecanismo de Slowly Changing Dimensions (SCD) Tipo 2.

5.3.1. Carga inicial das dimensões (clientes, produtos e vendedores).

O mesmo padrão de CTE foi aplicado às três dimensões com suporte a SCD Tipo 2. O mecanismo opera em três etapas encadeadas:

- **CTE S:** seleciona todos os registros da tabela de origem no schema Relational (ex.: Relational.customers). Essa CTE funciona como um espelho do estado atual da fonte transacional e serve de base para as comparações das etapas seguintes;
- **CTE UPD:** executa um UPDATE em Dimensional.dim_customer (ou dim_product / dim_seller). O filtro T.id_customer = S.id_customer AND T.validity_end_date IS NULL localiza, para cada ID de origem, o único registro que ainda está ativo, ou seja, aquele cuja validity_end_date ainda é NULL. Em seguida, uma segunda condição verifica se algum atributo rastreado mudou (ex.: customer_status, customer_name). Quando a mudança é detectada, validity_end_date recebe current_date, "fechando" aquela versão do registro. A

cláusula RETURNING T.id_customer devolve os IDs dos registros recém-encerrados, tornando-os disponíveis para a etapa seguinte;

- **INSERT final:** insere os novos registros na dimensão. A condição WHERE captura dois grupos: (1) os IDs retornados pela CTE UPD, que precisam de uma nova versão ativa,, e (2) os IDs que ainda não existem na dimensão, registros completamente novos. Em ambos os casos, validity_start_date recebe current_date (marcando o início da validade da nova versão) e validity_end_date recebe NULL (indicando que este é o registro atualmente válido).

5.3.2. Carga da tabela fato (mês a mês).

A tabela fact_sales é populada por meio de um INSERT INTO ... SELECT que consolida dados das tabelas transacionais e das dimensões analíticas. As vendas foram carregadas mês a mês (janeiro, fevereiro, março e demais meses) para possibilitar a simulação e verificação do SCD Tipo 2 entre as cargas. O SELECT da carga da fato opera com a seguinte lógica de mapeamento de atributos e chaves:

- **Mapeamento das métricas:** os atributos quantity, unit_price, total_price e discount são extraídos diretamente de Relational.sale_items iv, pois representam os dados de detalhe de cada item vendido, o nível de granularidade da tabela fato;
- **Mapeamento da seller_key:** obtida pelo INNER JOIN entre Relational.sales v e Dimensional.dim_seller vdd usando v.id_seller = vdd.id_seller AND vdd.validity_end_date IS NULL. O filtro validity_end_date IS NULL garante que a venda seja associada à versão do vendedor que estava ativa no momento da carga, e não a uma versão histórica encerrada;
- **Mapeamento da customer_key:** obtida pelo INNER JOIN entre Relational.sales v e Dimensional.dim_customer c usando v.id_customer = c.id_customer AND c.validity_end_date IS NULL. O mesmo filtro validity_end_date IS NULL assegura que a SK registrada na fato aponte para o status do cliente vigente no momento em que aquela venda foi carregada, princípio central do SCD Tipo 2;
- **Mapeamento da product_key:** obtida pelo INNER JOIN entre Relational.sale_items iv e Dimensional.dim_product p usando iv.id_product = p.id_product AND p.validity_end_date IS NULL. Novamente, o filtro validity_end_date IS NULL captura apenas a versão ativa do produto;
- **Mapeamento da time_key:** obtida pelo INNER JOIN entre Relational.sales v e Dimensional.dim_time t usando v.sale_date = t.time_date. O campo sale_date da tabela transacional é comparado diretamente ao campo time_date da dimensão tempo, localizando o registro que representa exatamente o dia em que a venda ocorreu, sem risco de duplicidade, pois a dimensão tem um registro único por data;
- **Filtro temporal:** a função date_part('MONTH', v.sale_date) extrai o número do mês de sale_date e filtra apenas as vendas do mês desejado (ex.: = 01 para janeiro, = 02 para fevereiro, > 03 para os demais meses).

5.3.3. Simulação de mudança de status e processamento de histórico (SCD Tipo 2).

Para demonstrar o funcionamento do versionamento SCD Tipo 2, foram simuladas duas alterações de status diretamente na tabela transacional Relational.customers:

- **Primeira alteração:** UPDATE Relational.customers SET customer_status = 'Gold' WHERE id_customer BETWEEN 1 AND 5, os clientes de ID 1 a 5 tiveram seu customer_status promovido para 'Gold' após a carga de janeiro;

- **Segunda alteração:** UPDATE Relational.customers SET customer_status = 'Platinum' WHERE id_customer = 3, o cliente de ID 3 foi promovido novamente para 'Platinum' após a carga de março.

A cada alteração, o bloco de carga das dimensões (CTE S + CTE UPD + INSERT) foi executado novamente. O mecanismo detecta a diferença em customer_status, encerra o registro anterior preenchendo sua validity_end_date com current_date e insere uma nova linha com validity_start_date = current_date e validity_end_date = NULL, preservando assim o histórico completo de versões de cada cliente.

5.3.4. Verificação de resultados e auditoria do SCD Tipo 2.

Ao longo do roteiro, consultas de auditoria foram executadas para validar o comportamento do SCD Tipo 2. Os SELECTs combinam fact_sales com dim_customer (e, quando necessário, com dim_time) via INNER JOIN pelas respectivas SKs, filtrando c.id_customer BETWEEN 1 AND 5. Essas consultas provam que:

- As vendas de janeiro apontam para as SKs antigas dos clientes 1 a 5, ou seja, para a versão em que customer_status ainda não era 'Gold'. O vínculo histórico é preservado na fato mesmo após a atualização na dimensão;
- As vendas de fevereiro em diante, realizadas após a atualização de status, apontam para a SK nova de cada cliente, refletindo o customer_status vigente no momento da carga daquele mês;
- Uma segunda consulta filtrando t.time_month = 3 confirma que, em março, nenhum dos clientes de ID 1 a 5 realizou compras, validando que a fato não registrou linhas para esse grupo naquele período.

Arquivo utilizado (1. Environments/2. OLAP/):

- 3. Script.sql.

5.4. Cubo.

Como o pgAdmin 4 não é um servidor de dados dimensional, foi criada uma tabela desnormalizada que une a tabela fato a todas as dimensões, formando o cubo analítico. Esse processo foi realizado por meio do arquivo 4. Cube.sql e é composto por três estruturas principais:

- O bloco SELECT escolhe apenas os atributos relevantes para análise, excluindo deliberadamente as Surrogate Keys (seller_key, customer_key, product_key, time_key), os IDs de origem (id_seller, id_customer, id_product) e os campos de versionamento (validity_start_date, validity_end_date). Os atributos selecionados são: customer_name, customer_state, customer_gender e customer_status de dim_customer; product_name de dim_product; time_date, time_day, time_month, time_year e time_quarter de dim_time; seller_name de dim_seller; e as métricas quantity, unit_price, total_price e discount de fact_sales;
- O bloco INTO Dimensional.sales_cube cria fisicamente a tabela de destino no schema Dimensional, consolidando em uma única estrutura plana todos os dados necessários para as análises no Power BI;
- O bloco FROM parte da tabela central Dimensional.fact_sales fv e conecta todas as dimensões por meio de INNER JOINS: dim_customer dc via fv.customer_key = dc.customer_key; dim_product dp via fv.product_key = dp.product_key; dim_time dt via

fv.time_key = dt.time_key; e dim_seller dv via fv.seller_key = dv.seller_key. O uso de INNER JOIN garante que apenas combinações válidas, onde todas as dimensões possuem um registro correspondente, compõem o cubo.

Arquivo utilizado (1. Environments/2. OLAP/):

- 4. Cube.sql.

5.5. KPI (indicador de performance).

Para medir a receita total da empresa JJBike em relação às metas mensais, foi criada a tabela Dimensional.kpi por meio do arquivo 5. KPI.sql. O script é composto por quatro estruturas principais:

- O bloco SELECT define as três colunas que compõem a tabela: (1) dt.time_month aliado como month, que representa o número do mês; (2) SUM(fv.total_price) aliado como total_revenue, que agrega a receita realizada somando o campo total_price de todos os registros de fact_sales pertencentes àquele mês; e (3) uma estrutura CASE dt.time_month que atribui um valor de meta fixo a cada mês (de R\$ 220.000 em janeiro a R\$ 270.000 em dezembro), aliada como target;
- O operador de conversão ::numeric, posicionado após o END do bloco CASE, força a coluna target a se consolidar como tipo numérico decimal exato, o mesmo tipo de total_price em fact_sales. Sem essa conversão, a coluna gerada dinamicamente pelo CASE poderia ser tratada como tipo inteiro pelo PostgreSQL, causando incompatibilidade de formato ao calcular percentuais de atingimento no Power BI;
- O bloco INTO Dimensional.kpi cria fisicamente a tabela de destino no schema Dimensional, armazenando os dados consolidados de receita realizada e meta por mês;
- O bloco FROM parte de Dimensional.fact_sales fv e realiza um INNER JOIN com Dimensional.dim_time dt usando fv.time_key = dt.time_key, que relaciona cada linha da fato ao seu respectivo registro na dimensão tempo, fornecendo o campo time_month necessário para o agrupamento. O GROUP BY dt.time_month consolida todas as vendas de cada mês em uma única linha, e o ORDER BY dt.time_month ASC organiza o resultado em ordem cronológica crescente.

Arquivo utilizado (1. Environments/2. OLAP/):

- 5. KPI.sql.

6. Artefatos - Power BI.

1. As tabelas do banco de dados *Dimensional*, criadas no pgAdmin 4, foram conectadas ao Power BI.

6.1. Relatórios.

1. Foram feitas as seguintes alterações nas tabelas:

- Em fact_sales, foi criada uma coluna chamada discount_percent (com duas casas decimais), com a seguinte fórmula: $\text{discount_percent} = \text{TRUNC}((\text{'dimensional fact_sales'}[\text{discount}] / \text{'dimensional fact_sales'}[\text{total_price}]) * 100, 2)$;
- Em dim_customer, foi criada uma coluna chamada full_location, com a seguinte fórmula:
- location_map =
SWITCH(

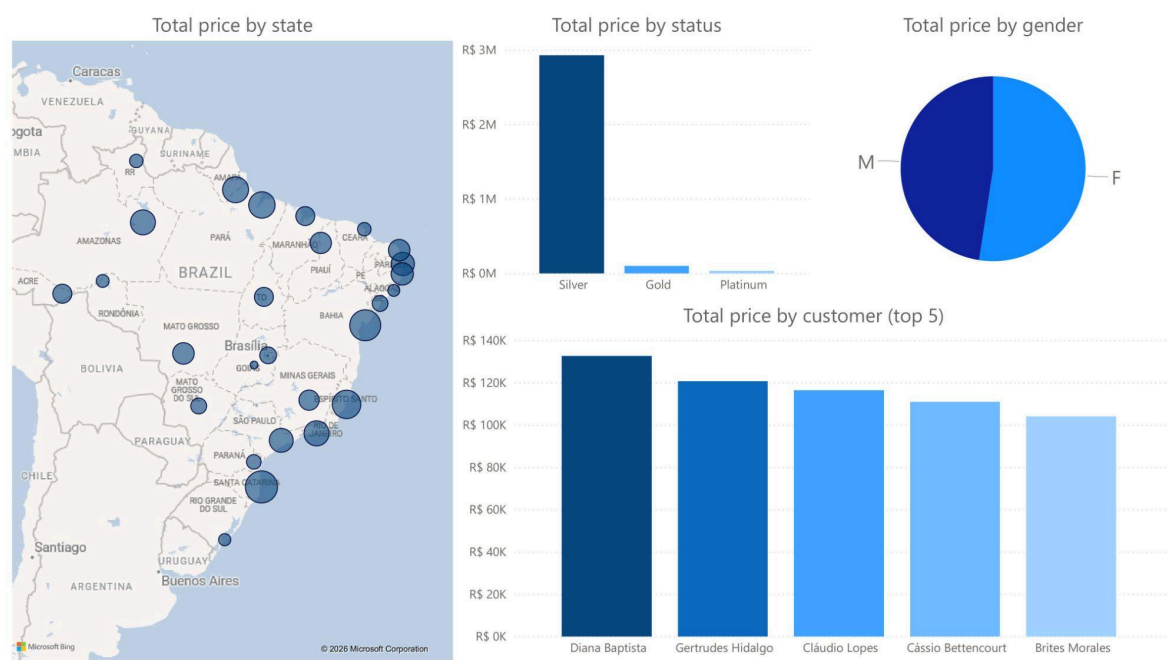

```
'dimensional dim_customer'[customer_state],
"AC", "Acre",
"AL", "Alagoas",
"AM", "Amazonas",
"AP", "Amapa",
"BA", "Bahia",
"CE", "Ceara",
"DF", "Federal District",
"ES", "Espirito Santo",
"GO", "Goias",
"MA", "Maranhao",
"MG", "Minas Gerais",
"MS", "Mato Grosso do Sul",
"MT", "Mato Grosso",
"PA", "Para",
"PB", "Paraiba",
"PE", "Pernambuco",
"PI", "Piauhi",
"PR", "Parana",
"RJ", "Rio de Janeiro",
"RN", "Rio Grande do Norte",
"RO", "Rondonia",
"RR", "Roraima",
"RS", "Rio Grande do Sul",
"SC", "Santa Catarina",
"SE", "Sergipe",
"SP", "Sao Paulo",
"TO", "Tocantins",
"Unknown"
) & ", Brazil".
```

- Em dim_kpi, foram criadas três métricas para que, no relatório de KPI, o card tenha comportamento dinâmico de acordo com o que estiver selecionado nos botões do button slicer:
 - measure_target =
 IF(
 ISFILTERED('dimensional kpi'[month]) || ISFILTERED('dimensional dim_time'[time_year]),
 SUM('dimensional kpi'[target]),
 0
).
 - measure_total_revenue =
 IF(
 ISFILTERED('dimensional kpi'[month]) || ISFILTERED('dimensional dim_time'[time_year]),
 SUM('dimensional kpi'[total_revenue]),
 0
).

2. Foram criados os seguintes relatórios:

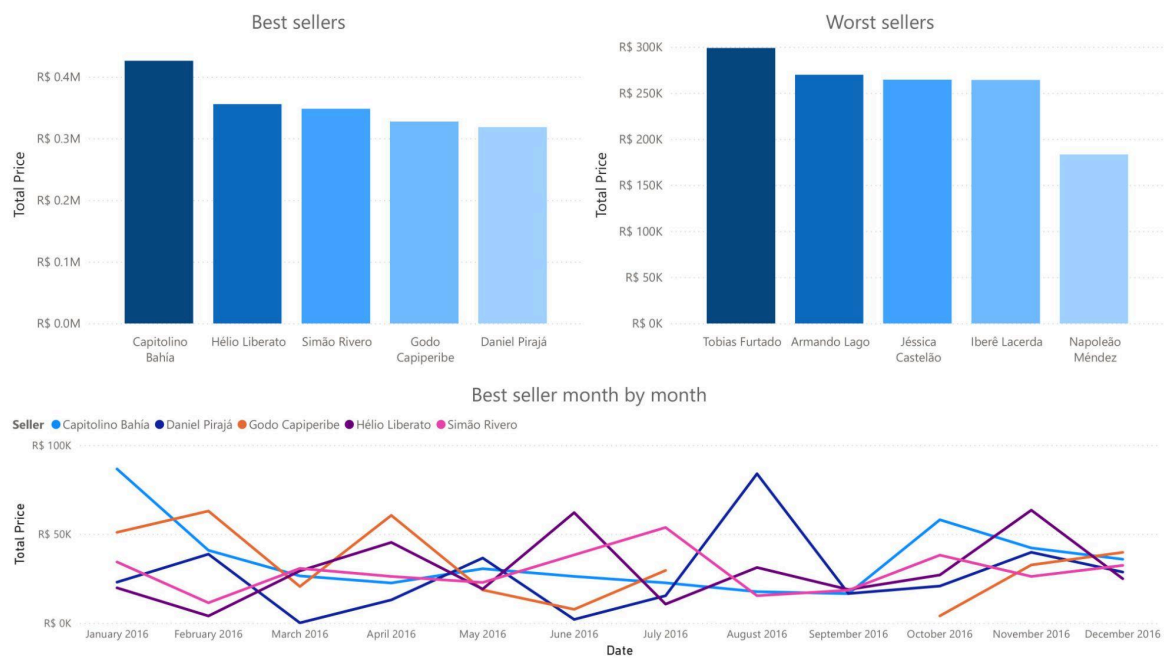
(Clientes)

O relatório de clientes apresenta: somatório de valor total por estado, somatório de valor total por status, somatório de valor total por gênero e somatório de valor total por cliente (top 5).



(Vendedores)

O relatório de vendedores apresenta: somatório de valor total por vendedor (top 5 melhores vendedores, top 5 piores vendedores e top 5 melhores vendedores comparados mês a mês).



(Produtos)

O relatório de produtos apresenta: somatório de valor total por produto (top 5 de produtos mais vendidos e top 5 de produtos menos vendidos) e somatório de desconto por produto (top 5 de produtos com mais desconto acumulado e top 5 de produtos com menos desconto acumulado).



6.2. Dashboards.

1. Foram criados os seguintes dashboards:

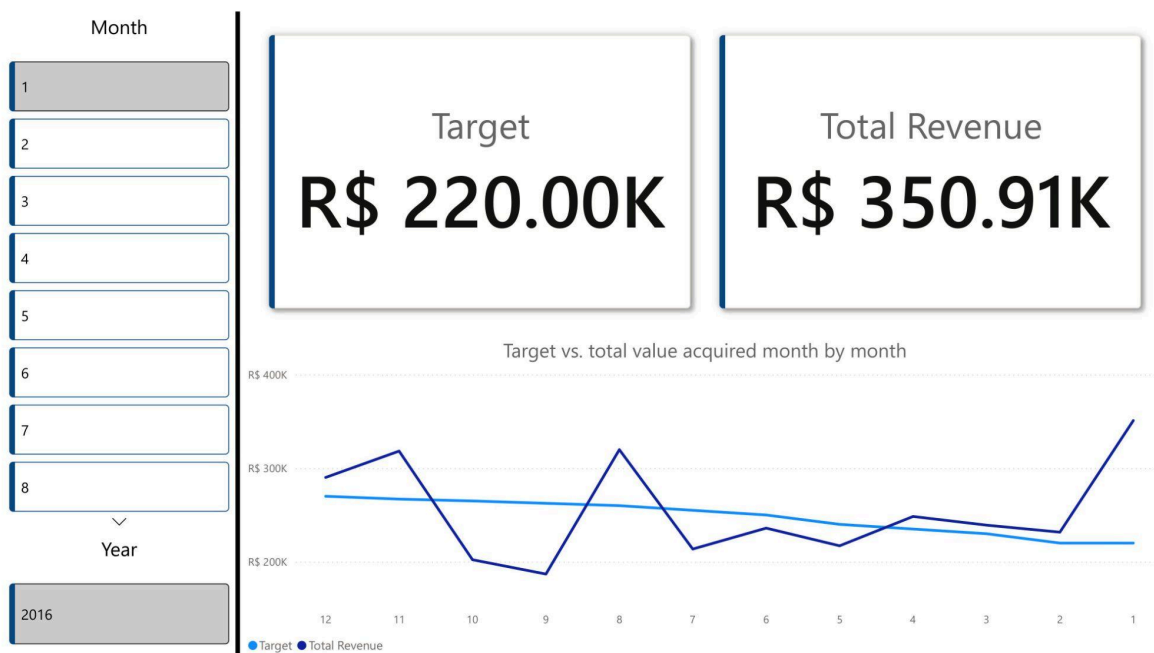
(Vendas)

O relatório de vendas apresenta: somatório de valor total de produtos vendidos (comparados mês a mês), somatório de quantidade de produtos vendidos (comparados mês a mês), somatório de desconto de produtos vendidos (comparados mês a mês) e cards de produtos, vendedores e clientes para filtragem.



(KPI)

O relatório de KPI apresenta: meta, valor total adquirido, somatório de meta e valor total adquirido (comparados mês a mês) e cards de mês e ano para filtragem.



6.3. Cubo.

1. Foram criadas as seguintes hierarquias a partir das dimensões:

- **Geographic Hierarchy:** customer_state → customer_name (dim_customer);
- **Time Hierarchy:** time_year → time_quarter → time_month → time_day (dim_time);
- **Observações:**

- customer_gender e customer_status devem ficar livres no painel lateral como Atributos de Dimensão Soltos, pois inseridos na Geographic Hierarchy implicaria que são parte de uma subdivisão geográfica;
- dim_seller e dim_product possuem apenas um atributo além de suas chaves, portanto não geraram hierarquias e seus atributos tornaram-se Atributos de Dimensão Soltos.

2. Representação visual do cubo criado:

Customer State	Quantity	Discount	Total Price	
AC		34	R\$ 11,172.65	R\$ 88,878.08
AL		6	R\$ 4,199.33	R\$ 26,817.61
AM		57	R\$ 15,881.10	R\$ 168,571.45
AP		53	R\$ 16,202.60	R\$ 183,945.80
BA		65	R\$ 18,104.59	R\$ 274,690.93
CE		11	R\$ 9,146.31	R\$ 37,562.17
DF		19	R\$ 14,839.02	R\$ 65,272.18
ES		89	R\$ 12,590.55	R\$ 226,925.84
GO		3	R\$ 858.20	R\$ 7,614.38
MA		38	R\$ 5,149.81	R\$ 87,431.06
MG		31	R\$ 18,853.70	R\$ 102,980.97
MS		19	R\$ 11,094.30	R\$ 57,340.96
MT		44	R\$ 11,279.89	R\$ 119,272.51
PA		82	R\$ 24,476.00	R\$ 190,925.91
PB		51	R\$ 8,791.16	R\$ 143,634.26
PE		42	R\$ 19,477.64	R\$ 129,473.93
PI		44	R\$ 20,851.47	R\$ 116,457.99
PR		22	R\$ 5,322.31	R\$ 45,718.33
RJ		57	R\$ 19,318.44	R\$ 169,913.05
RN		45	R\$ 16,227.19	R\$ 118,559.35
RO		13	R\$ 6,639.07	R\$ 33,213.18
RR		28	R\$ 7,962.75	R\$ 37,392.82
RS		12	R\$ 2,768.23	R\$ 28,466.43
SC		91	R\$ 35,968.90	R\$ 296,803.85
SE		15	R\$ 11,054.02	R\$ 56,151.97
SP		42	R\$ 17,043.43	R\$ 152,112.79
TO		47	R\$ 4,163.74	R\$ 88,034.93
Total		1060	R\$ 349,436.40	R\$ 3,054,162.73

Arquivos gerados disponíveis em '2. Artifacts/'.